Create a Neural network architecture from scratch in Python and use it to do multi-class classification on anydata. Parameters to be considered while creating the neural network from scratch are specified as:

(1) No of hidden layers : 1 or more

(2) No. of neurons in hidden layer: 100

(3) Non-linearity in the layer : Relu

(4) Usemore than 1 neuron in the output layer. Use a suitable threshold value Use appropriate Optimization algorithm

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X = iris.data
y = iris.target

scaler = StandardScaler()
X = scaler.fit_transform(X)

Y = np.eye(3)[y]

X_train, X_test, Y_train, Y_test, y_train, y_test = train_test_split(
    X, Y, y, test_size=0.2, random_state=42, stratify=y
)
```

In [2]:
```
np.random.seed(42)
W1 = np.random.randn(4, 100) * np.sqrt(2 / 4)
b1 = np.zeros((1, 100))
W2 = np.random.randn(100, 3) * 0.01
b2 = np.zeros((1, 3))

def relu(x):
    return np.maximum(0, x)

def softmax(x):
    x = x - np.max(x, axis=1, keepdims=True)
    exp_x = np.exp(x)
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)
```

In [6]:
```
lr = 0.01
epochs = 2000
m = len(X_train)
```

```
loss_history = []

for epoch in range(epochs):
    Z1 = np.dot(X_train, W1) + b1
    A1 = relu(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = softmax(Z2)

    loss = -np.mean(np.sum(Y_train * np.log(A2 + 1e-9), axis=1))
    loss_history.append(loss)

    dZ2 = (A2 - Y_train) / m
    dW2 = np.dot(A1.T, dZ2)
    db2 = np.sum(dZ2, axis=0, keepdims=True)

    dA1 = np.dot(dZ2, W2.T)
    dZ1 = dA1 * (Z1 > 0)
    dW1 = np.dot(X_train.T, dZ1)
    db1 = np.sum(dZ1, axis=0, keepdims=True)

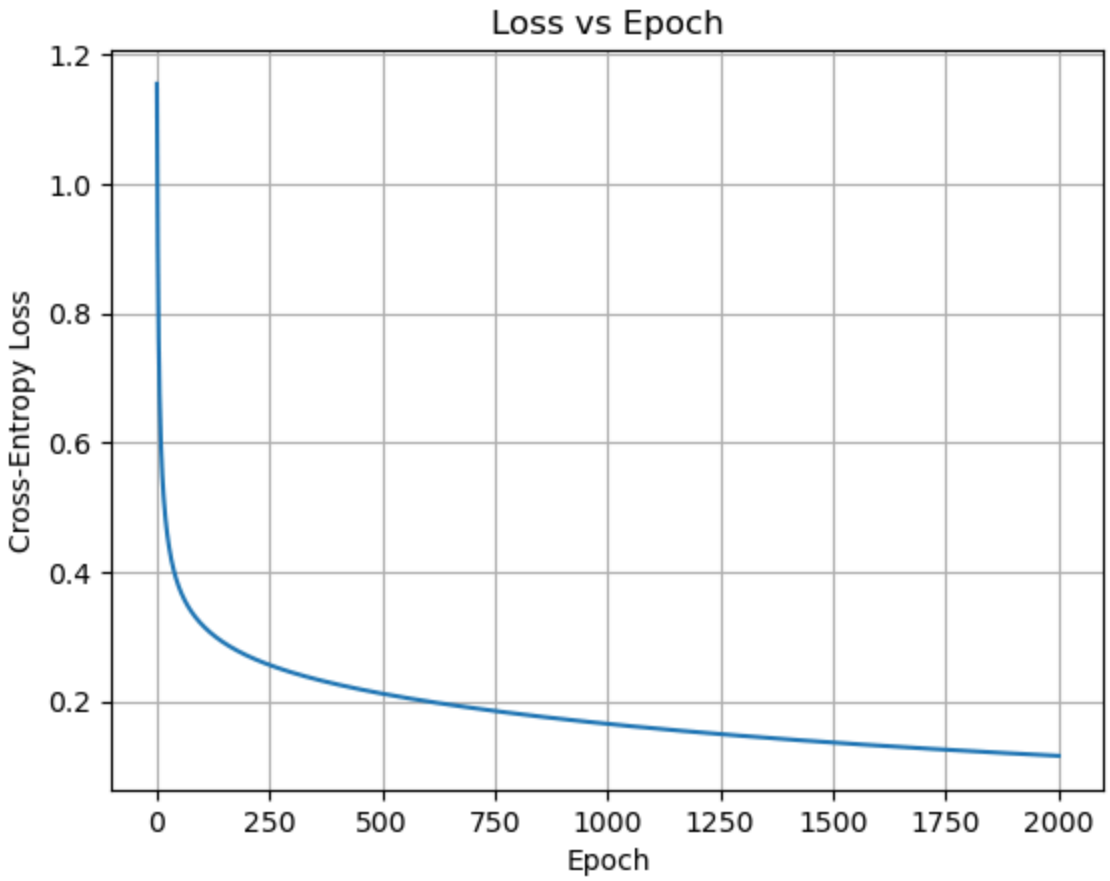
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1
    b1 -= lr * db1

    if (epoch + 1) % 500 == 0:
        print(f"Epoch {epoch + 1}, Loss: {loss:}")
```

```
Epoch 500, Loss: 0.10073415247019316
Epoch 1000, Loss: 0.08914041499519856
Epoch 1500, Loss: 0.08005806738407276
Epoch 2000, Loss: 0.07286918126882526
```

In [4]:

```
plt.plot(range(1, epochs + 1), loss_history)
plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.title("Loss vs Epoch")
plt.grid(True)
plt.show()
```



In [7]:
```
Z1 = np.dot(X_test, W1) + b1
A1 = relu(Z1)
Z2 = np.dot(A1, W2) + b2
probabilities = softmax(Z2)

y_pred = np.argmax(probabilities, axis=1)

print("\nTest Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_name
```

```
Test Accuracy: 93.33333333333333 %

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```

In [ ]: